The Transformer

From architecture blocks to MiniViT on MNIST

Dr. Papa-Séga WADE

AIMS Senegal | Session 2 <> Applied GAAI

MODULE 0 - Session 2: The Transformer architecture

"Attention is all you need, but understanding is what we build today"

Session 2 Objective

By the end of this session

Connect the attention intuition from Session 1 to Transformer blocks, architecture families, and a small Vision Transformer on MNIST.

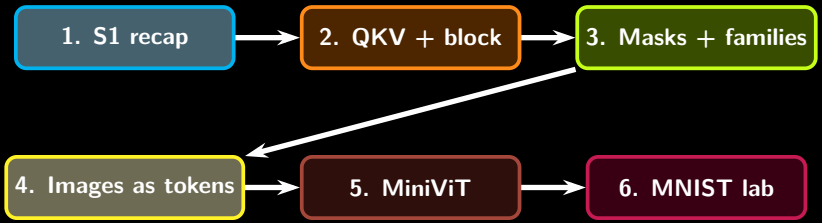
You should be able to explain

- where Transformers sit between NLP and Generative AI,
- how Q, K, V become an attention matrix,
- what residual connections, LayerNorm, and FFNs do,
- how BERT and GPT differ through masking,
- how an image becomes a sequence of patches.

What we do not do today

We do not repeat all of Session 1. We use it as a launchpad for architecture, masks, MiniViT, and the lab.

Session Roadmap



Pedagogical thread

We first position Transformers in the NLP/Generative AI landscape. Then we turn attention into an architecture and test the same idea on images.

Generative AI vs NLP

Natural Language Processing

NLP is the field of AI focused on human language: tokenization, classification, retrieval, translation, summarization, question answering, and sometimes text generation.

Not all NLP is Generative AI

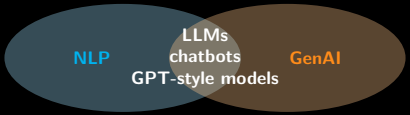
Sentiment analysis, named entity recognition, and parsing can be NLP without creating new content.

Generative AI

Generative AI models learn patterns in data and produce new content: text, code, images, audio, video, molecules, designs, or simulations.

The overlap

Large language models are the overlap: they are NLP systems trained with generative objectives.

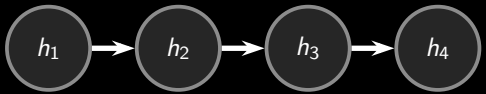


Transformers became the shared architecture behind much of this overlap.

Fast Recap: From Memory to Direct Access

RNN/LSTM viewpoint

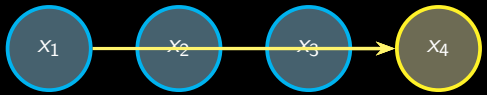
Information travels step by step through a hidden state. This was the bottleneck observed in Lab S1.



context moves through time

Transformer viewpoint

Each token directly compares itself with the tokens available in its context window.



context is directly accessible

Self-Attention in One Sentence

Self-attention builds a new representation of each token by taking a weighted average of the information carried by other tokens.

Example

Sentence: **"Seynabou is a brilliant computer scientist and mathematician."**

The token **Seynabou** should attend to **brilliant**, **computer scientist**, and **mathematician** to build a positive contextual representation.

Key idea

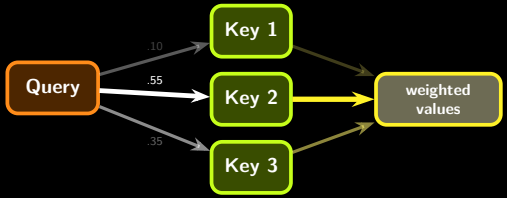
Attention is not just memory. It is **contextual selection**.

Q, K, V: The Search Analogy

Intuition

For each token, the model performs a small internal search:

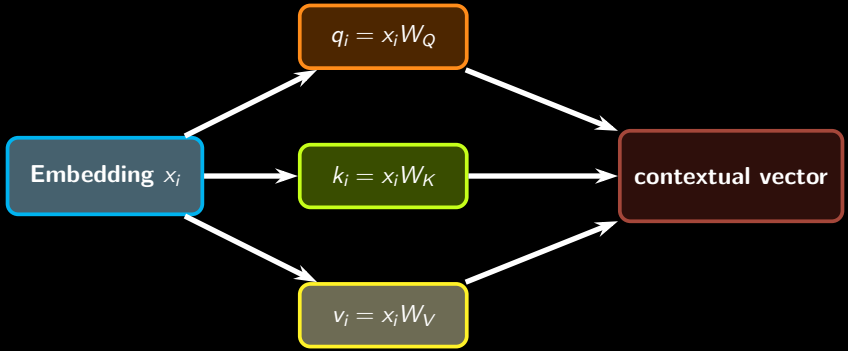
- **Query**: what am I looking for?
- **Key**: what does each token advertise?
- **Value**: what information will I copy if selected?



Analogy limit

This is not web search. It is learned vector matching inside the model.

From Embeddings to Q, K, V



Why three projections?

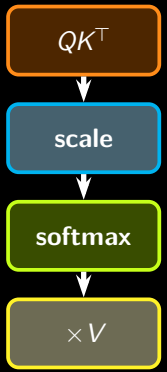
The same token must play different roles: asking a question, being matched by other questions, and providing information.

Scaled Dot-Product Attention

Compact formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top$: compatibility scores between tokens
- $\sqrt{d_k}$: scaling for numerical stability
- softmax: scores become weights
- V: values are mixed using those weights

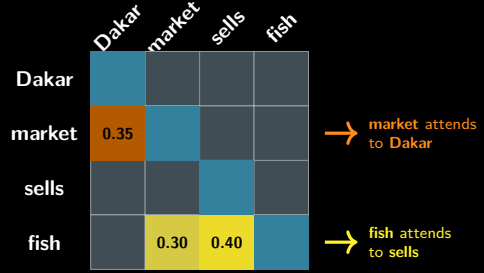


What an Attention Matrix Means

Reading the matrix

Each row is one token asking: **which tokens should I use to update myself?**

- rows sum to 1 after softmax
- brighter cells mean stronger attention
- different heads can show different patterns



Attention map: Senegalese context

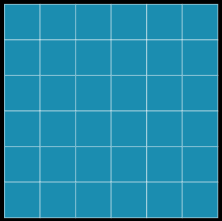
Masks: What Is Each Token Allowed to See?

BERT-style encoder

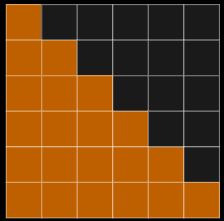
Bidirectional self-attention: every token can attend to every other token.

GPT-style decoder

Causal self-attention: token t can see the past and itself, but not future tokens.



full attention



causal mask

Same formula, different information flow: the mask changes model behavior.

Why Multiple Heads?

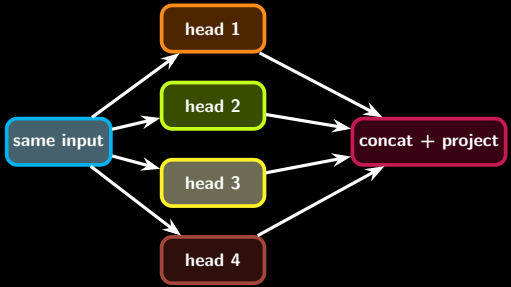
One head is one perspective

One attention head may learn a useful relationship, but language contains many relationships at once.

Multi-head intuition

Several heads let the model look from multiple angles simultaneously:

- syntax,
- local phrase structure,
- long-distance reference,
- punctuation or formatting.



Multi-Head Attention: Compact View

Mathematical summary

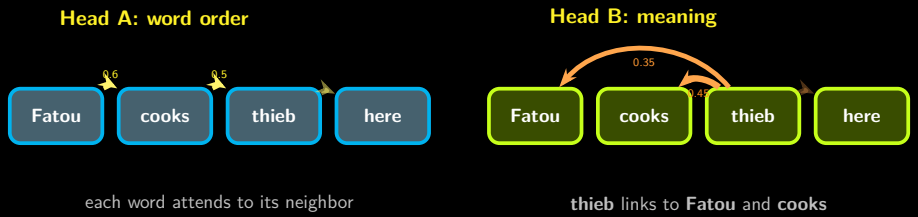
$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W_O$$

$$\mathrm{head}_i = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Important interpretation

The heads are not manually assigned roles. The model learns useful roles during training.

Attention Head Visualizations



Reading this diagram

Head A learned to track word order (syntax). Head B learned that **thieboudienne** is food that someone **cooks**. Different heads capture different relationships in the same sentence.

The Transformer Has No Built-In Order

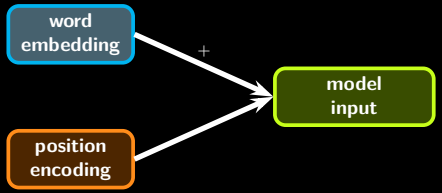
The problem: attention does not know word order

If we shuffle the words, the attention scores are the same. The model cannot tell which word came first.

Positional Encoding: Give Each Token a Seat Number

Analogy: seats in a classroom

Imagine students sitting in a classroom. Without seat numbers, you just see a group of people. With seat numbers, you know who sits first, who is in the middle, who is at the back.



The formula (for reference)

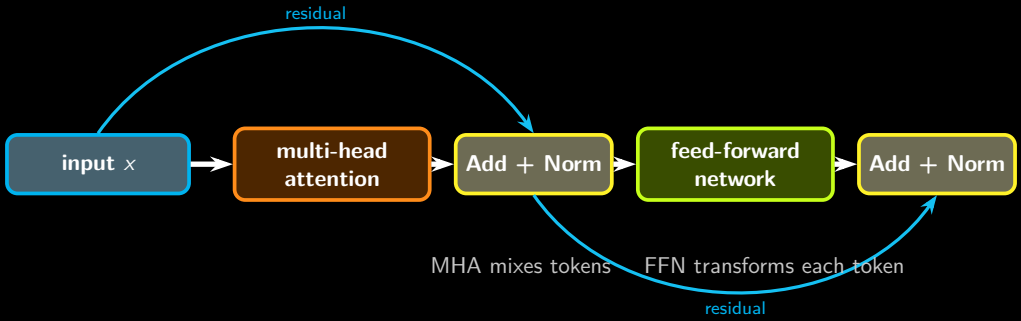
$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i{+}1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

What matters

- You do **not** need to memorize this formula.
- Key idea: each position gets a **unique pattern**.
- Nearby positions have **similar patterns**.
- Modern LLMs (GPT-4, LLaMA) use variants, but the principle is the same: **inject order**.

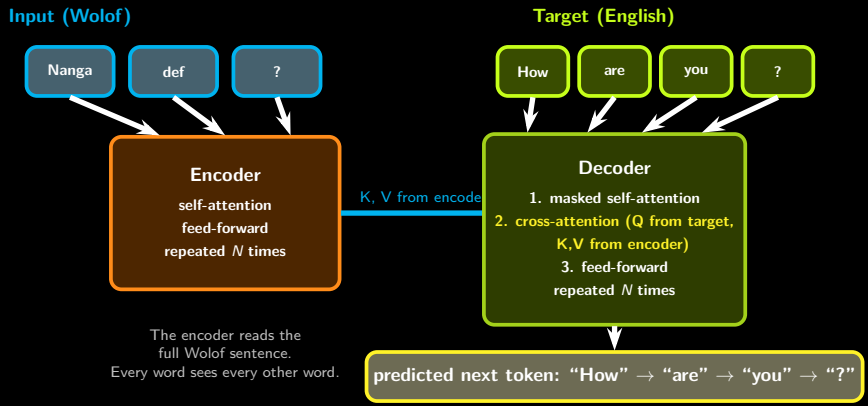
Transformer Block: The Reusable Unit



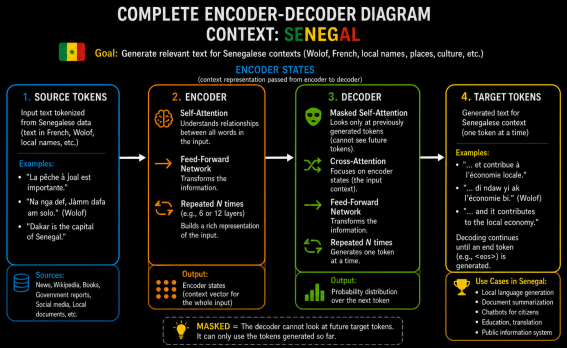
Plain-language description

Attention mixes information across tokens. The feed-forward network transforms each token independently. Residual connections stabilize deep stacking.

Complete Encoder-Decoder Diagram



Encoder-Decoder: Examples in Senegalese Context



How to read this diagram

The **encoder** reads the full input sentence at once. The **decoder** generates the output one token at a time, using what the encoder understood.

Senegalese examples

- **Wolof → French:**
  Encoder: "Nanga def ?"
  Decoder: "Comment" → "vas" → "tu" → "?"
- **Wolof → English:**
  Encoder: "Jere-jef"
  Decoder: "Thank" → "you"
- **Summarization:**
  Encoder: full news article in French
  Decoder: 2-sentence summary

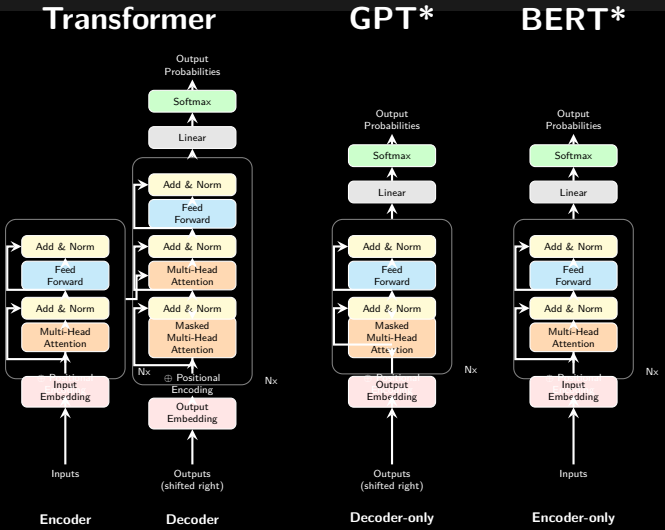
Encoder-Only, Decoder-Only, Encoder-Decoder

Family	Example	Typical use
Encoder-only	BERT	Classification, NER, sentence similarity, understanding tasks
Decoder-only	GPT	Autoregressive generation, chat, code, completion
Encoder-decoder	T5	Translation, summarization, text-to-text transformation

Quick reading heuristic

Understanding-only tasks often use encoders. Generation tasks use decoders. Sequence-to-sequence tasks often use encoder-decoder models.

Architecture Comparison: Transformer vs GPT vs BERT



*Illustrative example, exact model architecture may vary slightly

Why the Transformer Changed Everything

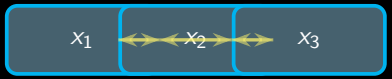
Parallelization

RNNs process tokens one after another. Transformers can compare many tokens in parallel during training.

Scaling

When architecture, data, and compute scale together, performance improves across many tasks.

sequential



parallel attention

From Text Tokens to Image Patches

Text Transformer

A sentence is **already** a sequence of tokens. Each word becomes one vector.

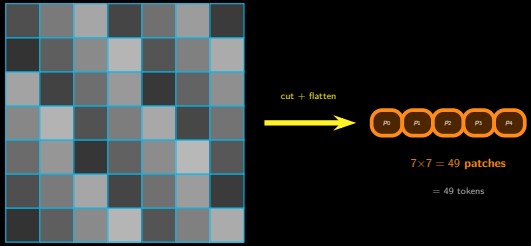
4 tokens (Wolof)

each token → one embedding vector
like a coordinate in $\mathbb{R}^d$

Vision Transformer idea

An image is **not** a sequence. We must create one:

1. Cut the image into 4×4 patches
2. Flatten each patch into a vector
3. Project it into an embedding
4. Now each patch = one token



28×28 MNIST digit

MiniViT Architecture for the Lab

Conv2d stride = patch size 49 patches + 1 CLS attention from scratch

Why CLS?

The CLS token collects information from all patches. Its final vector is used for classification.

Architecture Evolution in the Lab

Model	Inductive bias	What students should observe
MLP	none / flattened pixels	works, but ignores spatial structure
CNN	locality + translation reuse	strong on MNIST because digits are local strokes
CNN+LSTM	row sequence	useful when image rows behave like a sequence
MiniViT	global patch attention	learns relationships across distant regions

Key question: Does a more general architecture always win on a small structured dataset?

In-Class Lab: MNIST Architecture Evolution

Goal

Compare four ways to classify MNIST and implement the Transformer-specific pieces of MiniViT.

Notebook path

`lab2_mnist_transformer.ipynb`

1-hour afternoon scope

We focus on the MiniViT core first, then use the remaining time for interpretation.

- MLP: flattened pixels
- CNN: local spatial filters
- CNN+LSTM: image rows as a sequence
- MiniViT: patch tokens + attention

What Students Implement in MiniViT

Core code tasks

1. patch embedding with `Conv2d`,
2. self-attention from scratch,
3. Transformer block with residuals,
4. CLS token classification head.

Interpretation tasks

- Compare accuracy curves.
- Ask why CNN is strong on MNIST.
- Inspect whether CLS attention focuses on ink pixels.
- Explain when CNN+LSTM might beat MiniViT.

Lab Timing: Keep It Feasible

Recommended classroom pacing

For 1 hour, the goal is conceptual completion, not a perfect benchmark.

Time	Action
0–10 min	Load notebook, data, and utilities.
10–20 min	Review MLP/CNN/CNN+LSTM as reference architectures.
20–40 min	Implement MiniViT attention, patches, and CLS flow.
40–50 min	Run 1–2 epochs, depending on machines.
50–60 min	Compare accuracy, t-SNE, and attention maps qualitatively.

If machines are slow: fewer epochs, more architecture reading.

Evening Lab: Inspect Attention in a Real Model

Why a second lab tonight?

In class, students build a tiny Transformer-like model. Tonight, they inspect attention in a pre-trained multilingual model.

Evening notebook

`gaai_aims_courses/labs/lab_s2`

```python
from transformers import AutoTokenizer, AutoModel

name = "distilbert-base-multilingual-cased"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModel.from_pretrained(
    name, output_attentions=True
)
```

- Load `distilbert-base-multilingual-cased`.
- Visualize attention heads.
- Compare English, French, and Wolof transliteration.

Optional Extension: Manual Attention

Tonight or advanced groups

Compute attention on 3 tokens by hand:

$$QK^\top \rightarrow \frac{QK^\top}{\sqrt{d_k}} \rightarrow \text{softmax} \rightarrow \text{Attention}(Q, K, V)$$

Teaching note

This closes the loop: the same matrix operations appear in text attention and image-patch attention.

What to Remember

1. Session 1 gave the intuition: attention gives direct access to context.
2. A Transformer block adds residuals, normalization, and feed-forward transformations around attention.
3. Masks explain why BERT can read both directions and GPT must generate left to right.
4. A Vision Transformer turns an image into a sequence of patch tokens.
5. MiniViT on MNIST shows the tradeoff between inductive bias and generality.
6. Tonight's lab moves from building a tiny model to inspecting attention in a pre-trained model.

Key Resources

Annotated Transformer

Harvard NLP - The Annotated Transformer

Best for seeing code and math side by side.

Illustrated Transformer

Jay Alammar - The Illustrated Transformer

Best for visual intuition before implementation.

Questions & Discussion

You can now connect attention, masks, Transformer blocks, and MiniViT.

Dr. Papa-Séga WADE

papa-sega.wade@m4x.org /LinkedIn/papa sega wade

"From neurons to Transformers, the narrative should now feel continuous."